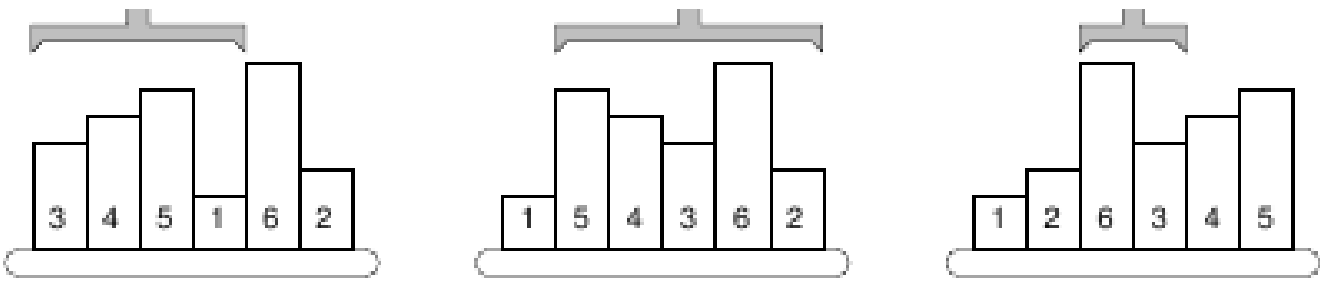


题目链接

本题是2007年ICPC欧洲区域赛中欧赛区的S题

题意

一个实验室里有 n 个长短不一的试管。你的任务是编写一段程序，用机器臂把它们按照高度从小到大的顺序排列。对于高度相同的试管，排序前后的相对位置应保持不变。排序方法如下图所示。



排序需要n 次操作，其中第i 次操作是“反转”序列i~Pi，其中Pi是目标状态中第i个试管当前所在的位置。比如，在上图中，初始时P1=4，因此反转试管1~4 就能把最左边的试管归位。类似地，第2 次操作前P2=6，因此反转试管2~6 就能把左数第2 个试管归位。

你的任务是输出 P1, P2, ..., Pn 的值，以便控制机器臂移动。注意i=Pi时实际上不需要反转，但仍然需要输出Pi。

分析

伸展树（splay）模板题，用懒标记表示反转提升效率，需要解决的一个难点：求特定结点的排名。给伸展树结点添加父指针字段，借助父指针可实现单次求特定结点排名的O(logn)时间复杂度算法。

说一个用伸展树解题时缩短运行时间的技巧：初始化时将伸展树构建成平衡二叉树。

AC代码

```
#include <iostream>
#include <algorithm>
using namespace std;

#define N 100050
int h[N], a[N] = {0}, c[N], n;
struct node {
    node *ch[2] = {NULL, NULL}, *fa = NULL; int s = 0, flip = 0;
    int cmp(int k) const {
        int d = k - ch[0]->s;
        return d == 1 ? -1 : d > 0;
    }
    void maintain() {
        s = ch[0]->s + ch[1]->s + 1;
    }
    void pushdown() {
        if (flip) {
            flip = 0;
            node *p = ch[0]; ch[0] = ch[1]; ch[1] = p;
            ch[0]->flip ^= 1; ch[1]->flip ^= 1;
        }
    }
} *null = new node(), *q[N], s[N];

node* rotate(node* o, int d) {
    node* k = o->ch[d^1];
    o->ch[d^1] = k->ch[d];
    k->ch[d]->fa = o;
    k->ch[d] = o;
    if (o->fa) o->fa->ch[o->fa->ch[1] == o] = k;
    k->fa = o->fa;
    o->fa = k;
    o->maintain();
    k->maintain();
    return k;
}

void splay(node*& o, int k) {
    o->pushdown();
    int d = o->cmp(k);
    if (d == -1) return;
    if (d) k -= o->ch[0]->s + 1;
    node* p = o->ch[d];
    p->pushdown();
    int d2 = p->cmp(k);
    if (d2 != -1) {
        splay(p->ch[d2], d2 ? k - p->ch[0]->s - 1 : k);
        if (d == d2) o = rotate(o, d^1);
        else o->ch[d] = rotate(o->ch[d], d);
    }
    o = rotate(o, d^1);
}

node* merge(node* left, node* right) {
    splay(left, left->s);
    left->ch[1] = right;
    right->fa = left;
    left->maintain();
    return left;
}

void split(node* o, int k, node*& left, node*& right) {
    splay(o, k);
    left = o;
    right = o->ch[1];
    right->fa = NULL;
    o->ch[1] = null;
    left->maintain();
}

int rnk(node* o) {
    int s = 0, t = 1; q[0] = o;
    while (o->fa) o = o->fa, q[t++] = o;
    while (t--) {
        q[t]->pushdown();
        if (!t || q[t-1] == q[t]->ch[1]) s += q[t]->ch[0]->s + 1;
    }
    return s;
}

node* build(int x, int l, int r) {
    node* o = q[x];
    o->ch[0] = l>1 ? build(x-1-(l-1>>1), l-1-(l-1>>1), l-1>>1) : null; o->ch[0]->fa = o;
    o->ch[1] = r ? build(x+r-(r>>1), r-(r>>1), r>>1) : null; o->ch[1]->fa = o;
    o->s = l+r; o->flip = 0; o->fa = NULL;
    return o;
}

void solve() {
    for (int i=1; i<=n; ++i) cin >> h[i], a[i] = h[i], c[i] = 0;
    sort(a+1, a+n+1);
    for (int i=1, j; i<=n; ++i) j = lower_bound(a, a+n+1, h[i]) - a, q[i] = s + j + c[j]++;
    node *root = build(n-(n>>1), n-(n>>1), n>>1), *o, *l, *r, *mid;
    for (int i=1; i<n; ++i) {
        int j = rnk(s+i);
        cout << j << ' ';
        if (j > i) {
            split(root, j, o, r);
            if (i > 1) {
                split(o, i-1, l, mid);
                mid->flip ^= 1;
                root = merge(merge(l, mid), r);
            } else o->flip ^= 1, root = merge(o, r);
        }
    }
    cout << n << endl;
}

int main() {
    ios::sync_with_stdio(false); cin.tie(0); cout.tie(0);
    while (cin>>n && n) solve();
    return 0;
}
```